Assignment 2 Report

Vinaykumar Reddy Junuthula

System Architecture

The experiments were conducted on the Lonestar6 supercomputer. Each compute node contains AMD EPYC 7763 processors with 64 cores per socket and two sockets per node, giving 128 cores per node. The architecture is x86_64 with two NUMA nodes. Each core has L1 and L2 caches, while a large shared L3 cache is available across cores. The system provides high memory bandwidth and is well suited for MPI-based parallel workloads.

Experiment Setup

Two MPI execution models were implemented: producer-consumer and work-pool.

In the producer-consumer model, rank 0 acts as a centralized broker with a bounded buffer. Producers send work messages to the broker, while consumers request work from it. This introduces centralized coordination and makes rank 0 a communication bottleneck.

In the work-pool model, all processes act as both producers and consumers. Each process sends work to random processes and consumes incoming work directly, eliminating centralized coordination.

Each program was executed with 4, 8, 12, and 16 processes. For each process count, five runs of 120 seconds were performed. Throughput was computed as the number of messages consumed per second. Figure 1 shows the average throughput, with error bars representing the minimum and maximum values across the five runs.

Performance Results

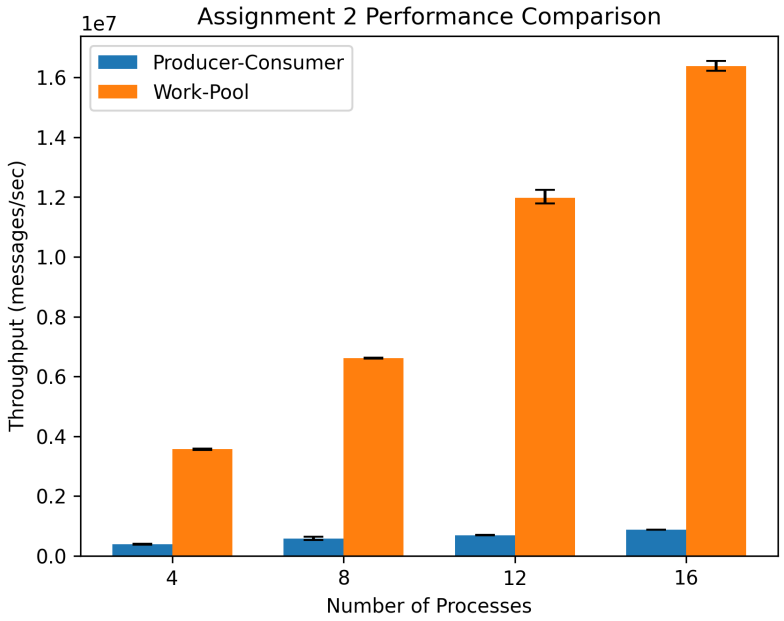

Figure 1: Average throughput for Producer-Consumer and Work-Pool models. Error bars show minimum and maximum throughput across five runs.

Observations

Throughput increases with the number of processes for both execution models, but the work-pool model consistently performs much better than the producer-consumer model.

At 4 processes, the producer-consumer model achieves about 0.39 million messages per second, while the work-pool model reaches about 3.57 million messages per second. At 16 processes, the producer-consumer model reaches about 0.87 million messages per second, whereas the work-pool model reaches about 16.38 million messages per second. This shows that the performance gap widens as the process count increases.

The producer-consumer model scales more slowly because rank 0 acts as a centralized broker and becomes a bottleneck as more producers and consumers are added. In contrast, the work-pool model distributes communication across all processes, allowing more simultaneous message exchanges and much better scalability.

The error bars show some run-to-run variability caused by communication overhead, scheduling differences, and system noise. Even with that variation, the overall trend is clear: the decentralized work-pool model provides substantially higher throughput and better scalability.